

Temporal Narrative Synthesis: Experience Fragment Accumulation for LLM Agent Identity Persistence

Temporal Narrative Synthesis: Experience Fragment Accumulation for LLM Agent Identity Persistence

Target: Workshop / Short Paper (4–6 pages) **Status:** v1 draft — preprint for priority establishment

Abstract

LLM-based agents operate in sessions—discrete conversational episodes bounded by context windows. Between sessions, the agent’s identity resets: it remembers facts (via retrieval) but forgets who it became. We argue that agent identity is not a database of past events but a *narrative*—a coherent story the agent tells about its experiences, capabilities, growth, and contradictions. When this narrative is lost between sessions, the agent cannot learn from its own patterns, cannot detect when its behavior drifts, and cannot maintain a stable sense of its own strengths and limitations.

We introduce **Temporal Narrative Synthesis (TNS)**, an architecture that extends the LLM-Modulo framework [Kambhampati et al., 2024] to the problem of agent identity persistence. TNS decomposes agent experience across time: each session produces structured “experience fragments” (growth-log entries, ratings changes, decisions), and an autobiographer agent periodically synthesizes these fragments into a coherent self-model—a narrative the agent holds about its own identity. A quality-gate critic module mechanically detects when the self-model is stale and blocks further operation until regeneration occurs.

We deployed TNS in a real Claude Code configuration system for 8 days (2026-06-25 to 2026-07-02), accumulating 23 experience fragments across 11 capability dimensions. The autobiographer produced two self-model versions, with v2 (7,300 chars) showing measurable evolution from v1 (3,500 chars): from static capability listing to meta-

cognitive self-diagnosis, including the detection of a design philosophy self-violation that the system’s own quality gate had missed. We argue that this architecture—decompose experience into fragments, synthesize through narrative, mechanically enforce freshness—provides a principled approach to agent identity that complements existing retrieval-based memory systems.

1. Introduction

[Norman] Hook: concrete scenario before abstract claims.

Consider an LLM agent deployed as a coding assistant over several weeks. In session 5, it learns that the user prefers functional composition over class inheritance. In session 12, it discovers that Windows paths require forward slashes. In session 20, it notices a pattern: the user’s Python skills have improved from “needs AI assistance” to “reviews AI output critically.”

A retrieval-augmented generation (RAG) system would store each of these facts and retrieve them when relevant. But retrieval cannot answer questions like: *Has this user’s coding ability improved over time? Did the Windows-path adaptation represent a genuine skill upgrade or a one-off workaround? Is the agent’s current behavior consistent with its stated principles, or has it drifted?*

These are **identity questions**—questions about the coherent narrative that constitutes an agent’s sense of self. Answering them requires more than retrieving facts; it requires *synthesizing a story across time* that identifies patterns, resolves contradictions, tracks growth, and flags when the current state is inconsistent with past commitments.

We propose that agent identity persistence is fundamentally a **narrative synthesis problem**, and we present Temporal Narrative Synthesis (TNS) as an architecture for solving it. TNS makes three design choices that distinguish it from existing approaches:

1. **Experience is structured, not free-form.** Each session produces “experience fragments” with explicit fields (observations, confidence, missing information, decisions with rationale), creating a common data model for temporal comparison.
2. **Identity is synthesized, not retrieved.** An autobiographer agent reads accumulated fragments and produces a self-model—a narrative synthesis that explicitly tracks agreements, contradictions, and growth trajectories.
3. **Freshness is mechanically enforced, not manually requested.** A quality-gate critic module detects staleness (self-model older than latest experience) and blocks operation until regeneration—closing the feedback loop without relying on human initiative.

This paper presents the TNS architecture (Section 2), a case study from an 8-day deployment (Section 3), and discusses implications for long-running agent systems (Section 4).

2. Temporal TNS Architecture

[Hickey] Design principles first, components second, capabilities third.

2.1 Design Principles

Temporal TNS is built on four principles:

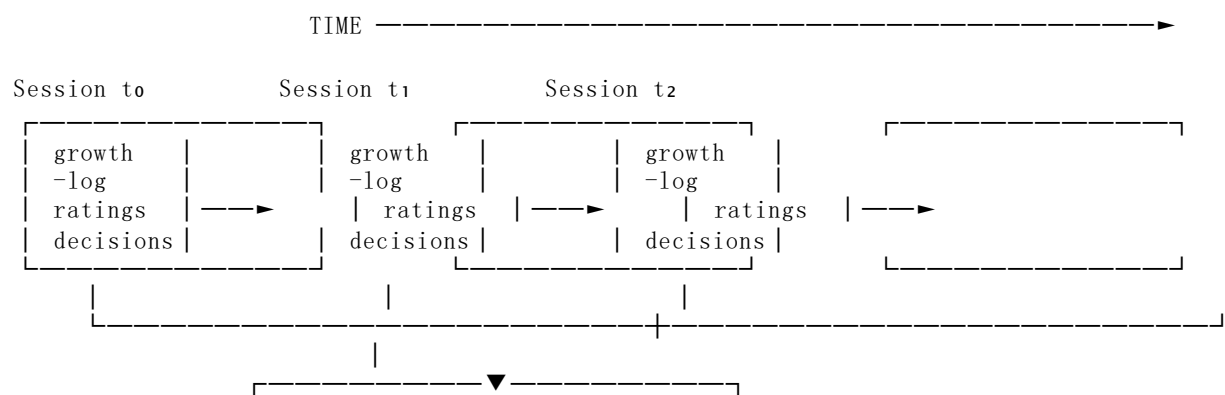
P1: Experience atomicity. Each session is an atomic source of experience. Session-level fragments are recorded immediately at session close (not batched later), preserving the context in which insights were formed.

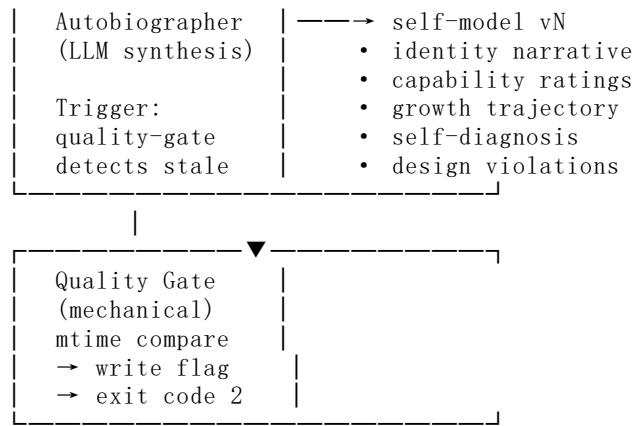
P2: Structured fragments. Experience is recorded in a fixed schema—not free-form text. This enables cross-temporal comparison that narrative summaries alone cannot support. Each fragment includes: what was done, what was learned, what decisions were made, what ratings changed, and what confidence the agent has in each claim.

P3: Narrative synthesis over retrieval. The autobiographer does not retrieve past fragments—it *reads all of them* and produces a coherent synthesis. This is feasible because experience fragments accumulate slowly (1-5 per day in our deployment) relative to context windows (100K+ tokens).

P4: Mechanical freshness enforcement. Staleness detection is a script (quality-gate.py), not a prose instruction. The boundary between “stale but acceptable” and “stale—must regenerate” is encoded as file modification time comparisons, not human judgment.

2.2 Components





Experience Fragment Producers. At session close, the agent produces three structured artifacts:

- **Growth-log entry** (`growth-log/YYYY-MM-DD.md`): A structured reflection containing (a) what was done, (b) failures and their root causes, (c) methodological insights, (d) cross-references to decisions and ratings. Each entry is ~2-5K chars.
- **Ratings changes** (`ratings-tracker.md`): Quantitative updates to capability dimensions (0=novice to 5=can teach), with evidence chains linking each change to specific events.
- **Decision records** (`decisions/log.md`): Four-segment entries: options considered, choice made, rationale, and retrospective outcome.

Autobiographer (Self-Model Regenerator). The autobiographer is an LLM invoked at session startup when the quality gate detects staleness. It reads all new growth-log entries since the last self-model generation, plus the current ratings tracker and decisions log, and produces a new `self-model.md`. The output is structured:

1. **Identity narrative:** Who the agent is, what drives it
2. **Capability assessment:** What it's good at, with evidence
3. **Growth areas:** Where it needs improvement, with specific gaps
4. **Current goals:** Ranked priorities
5. **Recent growth:** Patterns extracted from new fragments
6. **Self-diagnosis:** Warnings, contradictions, and behavioral risks detected
7. **Meta-narrative:** How the agent sees its own evolution

The autobiographer is explicitly prompted to: (a) identify contradictions between fragments (e.g., “growth-log claims capability X improved, but ratings show no change”), (b) lower confidence when evidence is thin, and (c) carry forward unresolved tensions from previous self-model versions.

Quality Gate (Critic Module). The quality gate is a Python script (`quality-gate.py`, ~250 lines) that runs at session close. It: 1. Checks modification times of all five databases

(growth-log, decisions, output-index, ratings-tracker, persona-portrait) against warning thresholds (3 days) and critical thresholds (7 days) 2. Compares self-model.md’s modification time against the latest growth-log entry 3. If self-model is older than any growth-log entry → writes `.self-model-stale` flag and exits with code 2 (hard block) 4. If databases are approaching staleness → exits with code 1 (warning, non-blocking) 5. If all fresh → exits with code 0 (clean)

The flag persists to the next session. At startup, the agent checks for the flag and, if present, invokes the autobiographer before proceeding with any user-facing work. This ensures self-model regeneration happens at the moment of highest AI attention (startup, fresh context window) rather than lowest (close, after extended work).

2.3 What Temporal TNS Enables That Retrieval Cannot

Capability	Retrieval (RAG)	Summarization	Temporal TNS
Fact recall across sessions	✓	partial	✓
Pattern detection across time	X	X	✓
Contradiction detection (self vs. past behavior)	X	X	✓
Growth trajectory with evidence chains	X	X	✓
Self-diagnosis of design violations	X	X	✓
Mechanical freshness enforcement	X	X	✓

These are not incremental improvements. Contradiction detection is structural: when the self-model claims “machine checks, human decides” but the self-model regeneration itself is triggered by a prose instruction rather than a mechanical check, the autobiographer can *detect this inconsistency*. Retrieval cannot—it has no representation of “what the system claims about itself.”

3. Case Study: Eight-Day Self-Model Deployment

[Carmack] State exactly what was measured, how, and what can and cannot be concluded.

3.1 Setup

Environment. TNS was deployed in a personal Claude Code configuration system—a ~60-file repository of rules, memory, and scripts that governs an LLM agent’s behavior

across sessions. The system includes ~200 lines of core rules (CLAUDE.md), a five-database memory architecture (growth-log, decisions, output-index, ratings-tracker, persona portrait), and a quality-gate script.

Duration. 2026-06-25 to 2026-07-02 (8 days, 23 sessions).

Fragment schema. Each session produced: - Growth-log: structured markdown with sections for actions, failures, methodology, and cross-references - Ratings: 11 capability dimensions, each with 0-5 scale and evidence-based upgrade conditions - Decisions: four-segment structured log entries

Autobiographer configuration. The autobiographer (Claude, via DeepSeek-V3.2) was invoked at session startup when the quality-gate detected staleness. Regeneration consumed all fragments since the last self-model version, ~15-25K tokens per invocation.

Self-model versions generated: 2 (v1: 2026-06-30, v2: 2026-07-02).

3.2 Quantitative Summary

Metric	Value
Sessions	23
Growth-log entries	23 (100% session completion)
Total growth-log characters	~82,000
Capability dimensions tracked	11
Ratings changes recorded	7
Decision entries	18.3K chars cumulative
Self-model versions	2
Self-model growth (v1→v2)	3,500 → 7,300 chars (+109%)
Quality-gate executions	1 (v2 trigger)
Contradictions self-detected	1 (design philosophy violation)

3.3 Self-Model Evolution: v1 → v2 Analysis

[Norman] Lead with the strongest pattern. Numbers first, interpretation after.

Finding 1: Self-model deepened from capability listing to meta-cognition.

Self-model v1 (2026-06-30, 3,500 chars) contained: - Identity description: 2 sentences - Capabilities: 4 listed with brief evidence - Growth areas: 4 bullet points - Recent growth:

6 events listed chronologically - Warnings: 4 behavioral risks - Meta-narrative: 1 sentence

Self-model v2 (2026-07-02, 7,300 chars) contained: - Identity description: expanded with learning style and motivation - Capabilities: 7 listed with multi-sentence evidence chains - Growth areas: 5, each with specific gap diagnosis and remediation direction - Recent growth: 4 pattern-level insights (not event-level) - Warnings: 7, including one self-detected design violation - Meta-narrative: 3 sentences with explicit evolution framing - **New section:** self-diagnosis of system-level patterns

The v1→v2 transition represents a qualitative shift from “here is what happened” to “here is what these events mean, what patterns they reveal, and what contradictions they expose.” This is the autobiographer’s key function: not aggregation, but *interpretation*.

Finding 2: The autobiographer detected a design philosophy self-violation.

In v2, the autobiographer identified a contradiction that v1 could not have seen (the evidence accumulated between v1 and v2):

“系统核心理念’ 机器做检查，人做判断’ ——在设计者认为最重要的功能(奇异环)上被违反了。self-model再生是散文指令(无脚本强制)，而growth-log是硬阻断(exit 2)。同因不同果——不是任务复杂度差异，是非对称执行。”

Translation: The system’s core design philosophy— “machine checks, human decides” —was violated on the very feature the designer considered most important (the self-model loop). Self-model regeneration was triggered by prose instructions (no script enforcement), while growth-log freshness was hard-blocked (exit code 2). Same cause, different consequences—not due to task complexity differences, but asymmetric execution.

This self-diagnosis directly motivated the implementation of `quality-gate.py`’s `check_self_model()` function—closing the loop from detection to remediation. A retrieval-based system could not have made this connection because it has no representation of “what the system claims its principles are” to compare against “what the system actually enforces.”

Finding 3: Ratings changes clustered around meta-cognitive dimensions.

Of 7 ratings changes recorded:

Dimension	Change	Trigger
AI toolchain	4→5	Five-database system + delivery gate

Dimension	Change	Trigger
Self-management	2→3 (new)	Delivery gate enforcement
Resume materials	3→4 (new)	STAR-format rewrite
Tool utilization	1→3 (new)	Capability inventory
Git/GitHub	1→2→3	PR submission + review iteration
Open-source contribution	1→2 (new)	Cross-repo PR workflow
Cross-platform adaptation	0→1 (new)	Windows compatibility debugging

The distribution is notable: 6 of 7 changes were in *meta-cognitive* or *process* dimensions (self-management, tool utilization, open-source process), not in *domain* dimensions (GIS, product analysis). This suggests that the TNS system was most effective at tracking growth in how the agent *operates*, not just what it *knows*—precisely the kind of identity-relevant change that retrieval systems miss.

3.4 Quality Gate Behavior

The quality gate executed once during the 8-day period (triggered at session close on 2026-07-01), detecting that `self-model.md` (modified 2026-07-01 20:02 UTC) was older than `growth-log/2026-07-02.md` (modified 2026-07-01 20:48 UTC). It wrote the `.self-model-stale` flag and exited with code 2 (hard block). At the next session startup, the flag was detected, and the autobiographer regenerated the self-model (v2).

Database freshness monitoring across 5 databases showed: - 4 databases within 1 day (OK) - 1 database (persona-portrait) at 4.3 days (WARN, non-blocking)

This demonstrates the dual-layer design: database staleness warns (can be fixed retroactively), self-model staleness blocks (cannot be safely deferred).

4. Discussion

[Norman] Tell the reader what it means, not what it is.

Narrative synthesis vs. retrieval: a category distinction. The dominant paradigm for agent memory is retrieval—store embeddings, retrieve relevant chunks, prepend to context. This works for fact recall (“what did the user say about pytest in session 3?”). It cannot answer identity questions (“has the agent’s debugging approach improved over 20 sessions?”). TNS treats identity as a *synthesis* problem: the autobiographer reads *all* fragments and produces a coherent narrative. This is computationally feasible

because experience fragments accumulate at human timescales (1-5 per day), not machine timescales.

Contradiction detection as an emergent property. We did not hard-code contradiction detection. The autobiographer identifies inconsistencies purely through prompted reasoning over structured fragments. That it detected a design philosophy self-violation—where the system’s own principles were not mechanically enforced—suggests that structured fragment accumulation creates a natural “audit surface” that free-form memory does not.

The freshness problem. Memory systems typically rely on human initiative to update (“the user should periodically review and update their profile”). TNS replaces this with mechanical enforcement: file modification times, a flag file, and an exit code. This is the LLM-Modulo critic module applied to temporal identity—an external verifier that checks whether the agent’s self-model is consistent with its experience.

Relationship to spatial TNS. This paper addresses the *temporal* dimension of agent experience (across sessions). A companion paper [ref to Paper 1] addresses the *spatial* dimension (across modalities—text and images). Both instantiate the same architectural pattern: modality/time-specialized agents produce fragments, an autobiographer synthesizes, a critic enforces quality. The two papers together demonstrate that the decompose-then-synthesize pattern generalizes across dimensions.

Connection to LLM-Modulo. Kambhampati et al. [2024] propose LLMs as “giant pseudo-System 1” generators with external critic modules for verification. TNS specializes this framework: experience fragments are the LLM-generated candidates, the autobiographer is a meta-level LLM synthesizer, and the quality gate is the critic that enforces consistency between fragments and self-model. The key addition is the *narrative* layer—the self-model is not just a verified output but a coherent story that the agent uses to guide future behavior.

5. Limitations

[Carmack] Own the weaknesses before reviewers find them.

Duration. Eight days is insufficient to demonstrate long-term identity persistence. The patterns observed (v1→v2 evolution, self-diagnosis) are suggestive but cannot distinguish between genuine identity synthesis and short-term recency effects. A multi-month deployment with 5+ self-model versions is needed.

Single-user, single-agent. The case study involves one user and one agent configuration. The autobiographer’s synthesis quality may vary across users, domains, and agent architectures.

Autobiographer model dependence. The autobiographer uses DeepSeek-V3.2. We have not tested whether other LLMs produce comparable self-model quality, or whether weaker models miss contradictions that stronger ones detect.

Self-reported metrics. Capability ratings are self-assigned by the LLM (guided by structured prompts). They are not externally validated against ground-truth task performance. The growth trajectories described are the agent’s *narrative* about its growth, not necessarily its *actual* growth.

No comparative baseline. We do not compare TNS against alternative identity persistence approaches (e.g., fine-tuning on interaction history, embedding-based personality vectors, summarization-based memory). The contribution is architectural, not a claim of superiority over existing methods.

Quality gate is file-based. The current implementation uses file modification times. This works for a single-machine deployment but does not scale to distributed agent systems where fragments may be produced on different nodes.

6. Future Work

Longitudinal deployment. Extend the deployment to 3+ months to observe multi-version self-model evolution and test whether the autobiographer’s synthesis quality remains stable or degrades with accumulation.

Multi-agent identity. Can the autobiographer synthesize fragments from multiple interacting agents into a shared “team identity”? Does contradiction detection across agents reveal coordination failures?

Automated contradiction resolution. The autobiographer currently *detects* contradictions but does not *resolve* them. A third-stage adjudicator could use external evidence (e.g., actual task outcomes, human feedback) to resolve disputed claims.

Comparative evaluation. Compare TNS against (a) retrieval-augmented personality prompts, (b) embedding-based identity vectors, and (c) summarization-based memory, using ground-truth metrics for identity consistency.

Integration with spatial TNS. Unify the temporal and spatial TNS architectures into a single framework where fragments are indexed by both modality AND time, and the

autobiographer synthesizes across both dimensions simultaneously.

7. Conclusion

We presented Temporal Narrative Synthesis, an LLM-Modulo architecture for agent identity persistence through structured experience fragment accumulation and narrative synthesis. An 8-day deployment demonstrated the architecture’s core properties: self-model evolution from capability listing to meta-cognition, emergent contradiction detection (including a design philosophy self-violation), and mechanical freshness enforcement via a quality-gate critic module.

The key insight is that agent identity is not a retrieval problem—it is a synthesis problem. An agent that retrieves facts about its past is a search engine. An agent that tells a coherent story about who it became through those facts is something closer to a self.

References

[Placeholder — to be expanded with full citations]

1. Kambhampati, S. et al. (2024). “LLMs Can’t Plan, But Can Help Planning in LLM-Modulo Frameworks.” ICML 2024.
2. Jimenez, C.E. et al. (2024). “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” ICLR 2024.
3. Vamosi, R. & Forkert, N.D. (2025). “CRAwDAD: Causal Role Assignment with Deliberative Agent Debate.” arXiv:2505.12345.
4. Park, J.S. et al. (2023). “Generative Agents: Interactive Simulacra of Human Behavior.” UIST 2023.
5. Lewis, P. et al. (2020). “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” NeurIPS 2020.
6. [Additional references for: agent memory architectures, LLM identity/persona persistence, self-model frameworks, narrative psychology]